

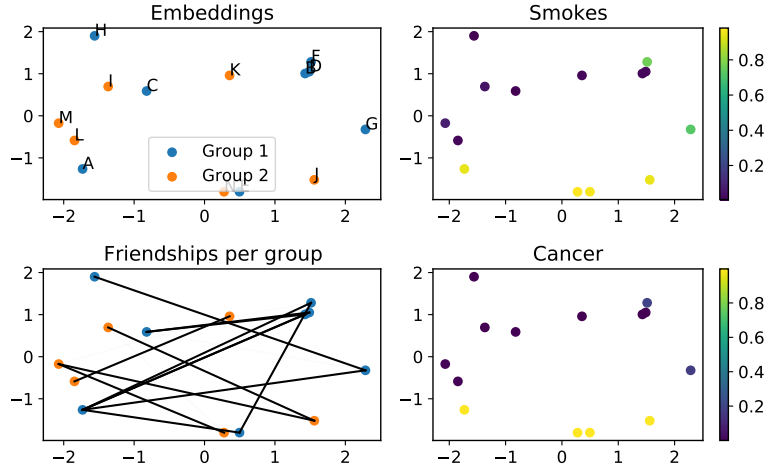


Figure 21: Smoker-Friends-Cancer example: learned embeddings showing the result of applying PCA on the individuals (top left); truth-values of smokes and cancer predicates for each embedding (top and bottom right); illustration of the friendship relations which are satisfied after learning (bottom left).

4.8. Reasoning in LTN

The essence of reasoning is to find out if a closed formula ϕ is the logical consequence of a knowledge-base $(\mathcal{K}, \mathcal{G}_\theta, \Theta)$. Section 3.4 introduced two approaches to this problem in LTN:

- By simply *querying after learning*²⁷ one seeks to verify if for the grounded theories that maximally satisfy $\mathcal{K}$, the grounding of ϕ gives a truth-value greater than a threshold q . This often requires checking an infinite number of groundings. Instead, the user approximates the search for these grounded theories by running the optimization a fixed number of times only.
- Reasoning by *refutation* one seeks to find out a counter-example: a grounding that satisfies the knowledge-base $\mathcal{K}$ but not the formula ϕ given the threshold q . A search is performed here using a different objective function.

We now demonstrate that reasoning by refutation is the preferred option using a simple example where we seek to find out whether $(A \vee B) \models_q A$.

Propositional Variables:

The symbols A and B denote two propositional variables.

Axioms:

$$A \vee B \tag{60}$$

²⁷Here, learning refers to Section 3.2, which is optimizing using the satisfaction of the knowledge base as an objective.

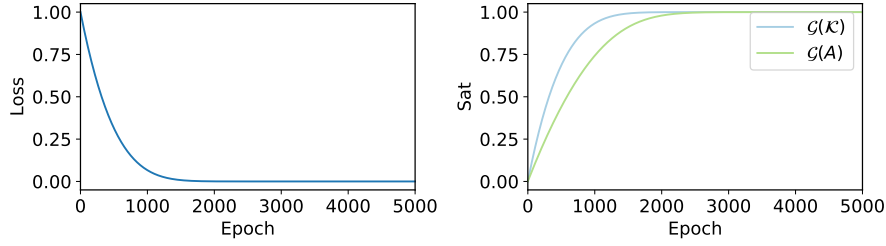


Figure 22: Querying after learning: 10 runs of the optimizer with objective $\mathcal{G}^* = \operatorname{argmax}_{\mathcal{G}_\theta} (\mathcal{G}_\theta(\mathcal{K}))$. All runs converge to the optimum $\mathcal{G}_1$; the grid search misses the counter-example.

Grounding:

$\mathcal{G}(A) = a, \mathcal{G}(B) = b$, where a and b are two real-valued parameters. The set of parameters is therefore $\theta = \{a, b\}$. At initialization, $a = b = 0$.

We use the probabilistic-sum S_P to approximate $\vee$, resulting in the following satisfiability measure:²⁸

$$\mathcal{G}_\theta(\mathcal{K}) = \mathcal{G}_\theta(A \vee B) = a + b - ab. \quad (61)$$

There are infinite global optima maximizing the satisfiability of the theory, as any $\mathcal{G}_\theta$ such that $\mathcal{G}_\theta(A) = 1$ (resp. $\mathcal{G}_\theta(B) = 1$) gives a satisfiability $\mathcal{G}_\theta(\mathcal{K}) = 1$ for any value of $\mathcal{G}_\theta(B)$ (resp. $\mathcal{G}_\theta(A)$). As expected, the following groundings are examples of global optima:

$$\mathcal{G}_1: \mathcal{G}_1(A) = 1, \mathcal{G}_1(B) = 1, \mathcal{G}_1(\mathcal{K}) = 1,$$

$$\mathcal{G}_2: \mathcal{G}_2(A) = 1, \mathcal{G}_2(B) = 0, \mathcal{G}_2(\mathcal{K}) = 1,$$

$$\mathcal{G}_3: \mathcal{G}_3(A) = 0, \mathcal{G}_3(B) = 1, \mathcal{G}_3(\mathcal{K}) = 1.$$

Reasoning:

$(A \vee B) \models_q A$? That is, given the threshold $q = 0.95$, does every $\mathcal{G}_\theta$ such that $\mathcal{G}_\theta(\mathcal{K}) \geq q$ verify $\mathcal{G}_\theta(\phi) \geq q$. Immediately, one can notice that this is not the case. For instance, the grounding $\mathcal{G}_3$ is a counter-example.

If one simply reasons by querying multiple groundings after learning with the usual objective $\operatorname{argmax}_{(\mathcal{G}_\theta)} \mathcal{G}_\theta(\mathcal{K})$, the results will all converge to $\mathcal{G}_1$: $\frac{\partial \mathcal{G}_\theta(\mathcal{K})}{\partial a} = 1 - b$ and $\frac{\partial \mathcal{G}_\theta(\mathcal{K})}{\partial b} = 1 - a$. Every run of the optimizer will increase a and b simultaneously until they reach the optimum $a = b = 1$. Because the grid search always converges to the same point, no counter-example is found and the logical consequence is mistakenly assumed true. This is illustrated in Figure 22.

Reasoning by refutation, however, the objective function has an incentive to find a counter-example with $\neg A$, as illustrated in Figure 23. LTN converges to the optimum $\mathcal{G}_3$, which refutes the logical consequence.

²⁸We use the notation $\mathcal{G}(\mathcal{K}) := \operatorname{SatAgg}_{\phi \in \mathcal{K}}(\mathcal{K}, \mathcal{G})$.

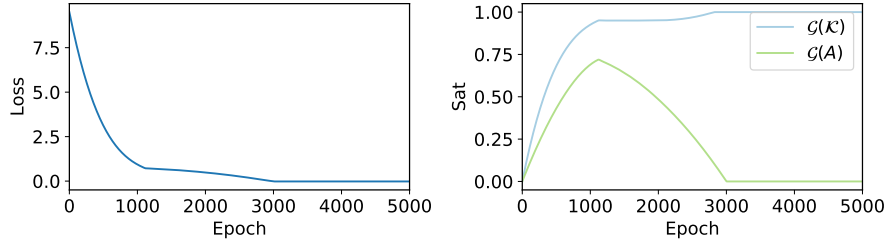


Figure 23: Reasoning by refutation: one run of the optimizer with objective $\mathcal{G}^* = \operatorname{argmin}_{\mathcal{G}_\theta} (\mathcal{G}_\theta(\phi) + \operatorname{elu}(\alpha, \beta(q - \mathcal{G}_\theta(\mathcal{K})))$, $q = 0.95$, $\alpha = 0.05$, $\beta = 10$. In the first training epochs, the directed search prioritizes the satisfaction of the knowledge base. Then, the minimization of $\mathcal{G}_\theta(\phi)$ starts to weigh in more and the search focuses on finding a counter-example. Eventually, the run converges to the optimum $\mathcal{G}_3$, which refutes the logical consequence.

5. Related Work

The past years have seen considerable work aiming to integrate symbolic systems and neural networks. We shall focus on work whose objective is to build computational models that integrate deep learning and logical reasoning into a so-called end-to-end (fully differentiable) architecture. We summarize a categorization in Figure 24 where the class containing LTN is further expanded into three sub-classes. The sub-class highlighted in red is the one that contains LTN. The reason why one may wish to combine symbolic AI and neural networks into a neurosymbolic AI system may vary, c.f. [17] for a recent comprehensive overview of approaches and challenges for neurosymbolic AI.

5.1. Neural architectures for logical reasoning

These use neural networks to perform (probabilistic) inference on logical theories. Early work in this direction has shown correspondences between various logical-symbolic systems and neural network models [27, 32, 52, 63, 65]. They have also highlighted the limits of current neural networks as models for knowledge representation. In a nutshell, current neural networks (including deep learning) have been shown capable of representing propositional logic, nonmonotonic logic programming, propositional modal logic, and fragments of first-order logic, but not full first-order or higher-order logic. Recently, there has been a resurgence of interest in the topic with many proposals emerging [13, 48, 53]. In [13], each clause of a Stochastic Logic Program is converted into a factor graph with reasoning becoming differentiable so that it can be implemented by deep networks. In [49], a differentiable unification algorithm is introduced with theorem proving sought to be carried out inside the neural network. Furthermore, in [11, 49] neural networks are used to learn reasoning strategies and logical rule induction.

Reasoning with LTN (Section 3.4) is reminiscent of this category, given that knowledge is not represented in a traditional logical language but in Real Logic.

5.2. Logical specification of neural network architectures

Here the goal is to use a logical language to specify the architecture of a neural network. Examples include [13, 24, 26, 56, 66]. In [26], the languages of extended logic programming (logic programs with negation by failure) and answer set programming are used as background knowledge to set up the initial architecture and set of weights of a recurrent neural network, which